

Digital Signal Processing with FAUST

Xianglei Wu



BACHELORARBEIT

eingereicht am

Fachhochschul-Bachelorstudiengang

Software Engineering

in Hagenberg

im Mai 2026

Betreuung:

Erik Pitzer, FH-Prof. DI (FH) Dr.

© Copyright 2026 Xianglei Wu

Diese Arbeit wird unter den Bedingungen der Creative Commons Lizenz *Attribution-NonCommercial-NoDerivatives 4.0 International* (CC BY-NC-ND 4.0) veröffentlicht – siehe <https://creativecommons.org/licenses/by-nc-nd/4.0/>.

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt. Die vorliegende, gedruckte Arbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Hagenberg, am 25. Mai 2026

Xianglei Wu

Inhaltsverzeichnis

Erklärung	iv
Preface	viii
Abstract	ix
Kurzfassung	x
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Forschungsfrage und Zielsetzung	2
1.3 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Grundlagen der Audio-DSP	4
2.1.1 Physikalische Grundlage des Audiosignals	4
2.1.2 Abtastung und Quantisierung	4
2.1.3 Signal-Rausch-Verhältnis (SNR)	5
2.1.4 Digitale Audiosignale als Sample-Folgen	6
2.2 Frequenzanalyse mit DFT und FFT	6
2.2.1 Zeitbereich und Frequenzbereich	6
2.2.2 Diskrete Fourier-Transformation	7
2.2.3 Fast Fourier Transformation	8
2.2.4 Amplituden- und Leistungsspektrum	9
2.2.5 Frequenzauflösung	9
2.3 Kurzzeitanalyse von Sprachsignalen	10
2.3.1 Nichtstationarität von Sprache	10
2.3.2 Frame-basierte Verarbeitung	11
2.3.3 Short-Time Fourier Transform	11
2.3.4 Magnitude und Phase	11
2.4 Theorie der Spectral Subtraction	11
2.4.1 Ziel und Grundidee des Verfahrens	12
2.4.2 Additives Modell von Sprache und Rauschen	12
2.4.3 Grundablauf der Spectral Subtraction	13
2.4.4 Voraussetzungen	14
2.5 Fensterung und spektrale Eigenschaften	15

2.5.1	Blockbildung und Randdiskontinuitäten	15
2.5.2	Spectral Leakage	15
2.5.3	Hanning- und Hamming-Fenster	15
2.6	Verarbeitungsschritte der Spectral Subtraction	16
2.6.1	Rauschschätzung in sprachfreien Abschnitten	16
2.6.2	Subtraktion des Rauschspektrums	17
2.6.3	Half-Wave Rectification	18
2.6.4	Reduktion des Restrauschens	18
2.6.5	Erhalt oder Nutzung der Phaseninformation	19
2.7	Rekonstruktion des bearbeiteten Signals	20
2.7.1	Inverse FFT	20
2.7.2	Überlappende Frames	21
2.7.3	Rekonstruktion des Zeitsignals	21
2.8	Echtzeitanforderungen	21
2.8.1	Latenz	21
2.8.2	Puffergröße	21
2.8.3	Rechenzeit pro Frame	22
2.9	Zusammenfassung	22
3	Faust: Functional Audio Stream	23
3.1	Einführung	23
3.2	Programmmodell	23
3.3	Syntax: Blockdiagramm-Komposition	24
3.4	Semantik	24
3.5	Compiler-Architektur	24
3.6	Codegenerierung	25
3.6.1	Skalarer Modus	25
3.6.2	Vektor-Modus	26
3.6.3	Parallel-Modus	26
3.6.4	Generierte Code	26
3.7	Backends und Architekturdateien	26
3.8	Performance	26
3.9	Zusammenfassung	27
4	Mathematical Elements	28
5	Using Literature	29
6	Printing the Manuscript	30
7	Closing Remarks	31
A	Technical Details	32
B	Supplementary Materials	33
B.1	PDF Files	33
B.2	Media Files	33

Inhaltsverzeichnis	vii
C Questionnaire	34
D LaTeX Source Code	35
Quellenverzeichnis	36
Literatur	36
Online-Quellen	36

Preface

Abstract

This should be a 1-page (maximum) summary of your work in English.

Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. ...

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Die digitale Signalverarbeitung (Digital Signal Processing, DSP) ist ein wichtiges Anwendungsfeld in der Informatik besonderen Wert in Telekommunikation, Audiotechnik, Sprachverarbeitung und Medizintechnik, die mit hohen Anforderungen an Laufzeiteffizienz, deterministische Latenz und numerische Präzision. Grundsätzlich sind die Implementierungen der DSP-Algorithmen in C oder C++, welches aufgrund deren direkten Zugriff auf Hardware Ressourcen und deren extrem effizienten Übersetzer (GCC, Clang) hochgradig optimierte Maschinencode erzeugt, für die Bachelorarbeit die erste Wahl. Der Knackpunkt trotz all den Vorteilen liegt an der doch noch sehr zeitintensive, fehleranfällige mathematische Übersetzung bis hin zu manuelle Implementierung von DSP-Algorithmen und das exportieren auf verschiedenen Zielplattformen, wie auf Desktop, verschiedensten Embedded Systemen, Web und Plug-in-Hosts; Daraus kristallisierten sich in den letzten zwei Jahrzehnten domänenspezifische Sprachen (Domain-Specific Languages, DSLs) für die Audio-Signalverarbeitung, welches durch Abstraktion und Code-Generierung die Probleme bewältigen soll.

Die domänenspezifische Sprache Faust (Functional Audio Stream), entwickelt am GRAME in Lyon von, verspricht einen Lösungsansatz zu den erwähnten Herausforderungen. Faust ist eine funktionale, blockbasierte Programmiersprache, die speziell für Audio-DSP entworfen wurde. Programme werden als algebraische Komposition von Signalverarbeitungsoperatoren formuliert. Der Faust-Compiler übersetzt die mathematisch knappe Beschreibung eines Signalflussdiagramms in mehrere Zielsprache, darunter C++, Rust, JavaScript und WebAssembly [9]. Aus einer einzigen Quellbeschreibung lassen sich somit Plug-ins für Digital Audio Workstations, Embedded-Anwendungen auf Mikrocontrollern und Webapplikationen erzeugen. [Vgl. 5, S. 2-6]

Um für die industrielle Tauglichkeit muss der durch Faust generierte Code sowohl funktional korrekt, als auch laufzeiteffizient sein. Befürworter von Faust argumentieren, dass der generierte Code durch globaler Optimierung über den vollständigen Signalflussgraphen, automatischer Vektorisierung [Vgl. 7, S. 2, S. 10-13] und Parallelisierung [Vgl. 2, S. 2-3] mit handgeschriebener C++ Code ebenbürtig oder dieses sogar übertreffen kann. Diese Behauptung vertreten auch die Entwickler in der offiziellen Faust-Dokumentation. [Vgl. 5, S. 6, S. 29] Ob handgeschriebener, manuell optimier-

ter C++-Code grundsätzlich performanter ist als Faust-generierter Code, ist in der wissenschaftlichen Literatur bislang nur punktuell untersucht. Eine systematische Literaturrecherche zeigt zwei peer-reviewte Studien mit direktem Performanz-Vergleich zwischen Faust-generiertem und handgeschriebenem C++-Code.

[**orlarey:hal-02159014freewerb**] berichten für den Reverb-Algorithmus *Freeverb* einen Geschwindigkeitsvorteil zugunsten von Faust von 28 Prozent. [Vgl 4, S. 5-6] erweitern diesen Vergleich auf neun Instrumente des *Synthesis Toolkit* (STK) und ermitteln dort Geschwindigkeitsvorteile zwischen 18 und 80 Prozent. Beide Studien stammen aus dem erweiterten Faust-Umfeld und untersuchen ausschließlich Algorithmen im Zeitbereich (Reverbs, Delays, physikalische Modelle).

Für FFT-intensive DSP-Algorithmen liegt keine vergleichende Untersuchung vor. Diese Lücke ist nicht zufällig: Die Faust-Dokumentation räumt selbst ein, dass die Sprache durch ihr sample-orientiertes Berechnungsmodell strukturelle Schwierigkeiten mit Multi-Rate-Algorithmen wie FFT und Faltung aufweist [5]. Da gerade FFT-basierte Verarbeitung den Kern vieler praxisrelevanter DSP-Anwendungen –etwa Sprachverbesserung, Audio-Codecs und akustische Analyse –bildet, ergibt sich hier eine wissenschaftlich wie praktisch relevante Forschungslücke.

Die vorliegende Arbeit setzt an dieser doppelten Lücke an: Sie untersucht systematisch und mit differenzierter Methodik, wie sich Faust-generierter C++-Code gegenüber einer manuell implementierten C++-Lösung verhält, wenn beide Implementierungen einen FFT-basierten Algorithmus realisieren. Als Anwendungsfall wird ein klassisches, in der Literatur umfassend dokumentiertes Verfahren der Einkanal-Rauschunterdrückung herangezogen – *Spectral Subtraction* nach Boll. [1] Die Wahl eines etablierten, wohlverstandenen Algorithmus erlaubt es, den Fokus konsequent auf den Sprach- und Implementierungsvergleich zu legen.

1.2 Forschungsfrage und Zielsetzung

Ziel dieser Arbeit ist es an dieser doppelten Lücke anzusetzen. Sie soll die Untersuchung der Eignung von Faust für die Implementierung von FFT-intensiven Echtzeit-DSP-Anwendungen im direkten Vergleich zu einer äquivalenten manuellen nicht optimierten C++-Implementierung zeigen. Dabei soll die Handhabung und Erlernbarkeit der Sprache Faust bei der Implementierung berücksichtigt werden.

Zur Beantwortung der Hauptaufgabe werden folgende Teilfragen untersucht:

- Wie groß ist der Performanz-Unterschied(CPU-Zeit pro Audio-Block) zwischen Faust-generiertem und handgeschriebenem Code für elementare DSP-Bausteine(Gain, Biquad-Filter, FIR, FFT)?
- Wie skaliert dieser Unterschied bei der Verknüpfung zu einer vollständigen Pipeline(Framing, Fensterung, FFT, Spectral Subtraction)?
- Bestehen Objektive Qualitätsunterschiede(PESQ, STOI, SNR-Verbesserung) zwischen den beiden Implementierungen bei identischem Algorithmus?
- Developer Experience: Wie bewährt sich Faust hinsichtlich Entwicklerfreundlichkeit, Codelänge, Wartbarkeit, Komplexität im Vergleich zum Aufbau komplexer Datenstrukturen in C++?

1.3 Aufbau der Arbeit

Die vorliegende Bachelorarbeit ist methodisch in acht Kapitel strukturiert. Der Prozess umfasst, theoretische Grundlagen der DSP, Aufbau des Prototyps, die Durchführung systematischer Vergleiche sowie die abschließende Evaluierung.

Nach dieser Einleitung wird im Kapitel 2 (die Grundlagen) für das notwendige Verständnis der Digitale Signalverarbeitung erläutert, genaueres über Audio-DSP, Fast Fourier Transformation, Fensterfunktionen und sowie generell Echtzeitanforderungen.

Darauf aufbauend stellt Kapitel 3 (Faust) die domänenspezifische Sprache im Detail vor. Einblicke in die funktionale Syntax und Semantik sowie die zugrundeliegende Compiler-Architektur. In Kapitel 4 (Spectral Subtraction) wird die ausgewählte Algorithmus, ein Rauschunterdrückungsverfahren nach Boll theoretisch hergeleitet und auf deren Parametern genauer eingegangen.

In Kapitel 5 (Implementierung) wird die praktische Umsetzung beider Varianten beschrieben. Faust-Code, C++-Code, gemeinsame Schnittstellendesign und sowie Build-System für eine faire Vergleichsbasis werden dokumentiert.

Die Beantwortung der Forschungsfragen erfolgt in Kapitel 6 (Evaluation). In diesem Kapitel werden die Versuchsaufbau detailliert dargelegt und die Ergebnisse der Mikro- und Makro-Benchmarks (Laufzeit, Speicher, Binärgröße) sowie der Qualitätsmessungen (PESQ, STOI) auf beiden Testplattformen ausgewertet.

Danach wird in Kapitel 7 (Diskussion) die erhobene Daten interpretiert. Es werden Limitationen der Untersuchungen, die Aussagekräftigkeit der Durchführungen und die qualitativen Aspekte der Entwicklererfahrung beider Sprachen gegenübergestellt.

Zum Abschluss mit Kapitel 8 (Fazit und Ausblick) wird die Arbeit zusammengefasst und ein Ausblick auf mögliche Erweiterungen, oder Lücken hingewiesen.

Kapitel 2

Grundlagen

2.1 Grundlagen der Audio-DSP

2.1.1 Physikalische Grundlage des Audiosignals

Das Audiosignal ist physikalisch eine sich in einem Medium ausbreitende Druck- und Bewegungsstörung, die durch ihre zeitlichen und räumlichen Schwingungsgrößen beschrieben wird.

Die einfachste Darstellung eines Signals im Zeitbereich ist die Sinuswelle,

$$x(t) = A \cdot \sin(\omega t + \phi) \quad (2.1)$$

also eine zeitliche Schwankung des Drucks um einen Mittelwert [Vgl. 6, S. 504].

Die wichtigsten Größen im Zeitbereich sind:

- **Amplitude** A : die Größe der Schwingung, also die maximale Druckabweichung vom Ruhezustand.
- **Frequenz** f : die Anzahl der Schwingungswiederholungen pro Sekunde, gemessen in Hertz (Hz).
- **Periode** T : die Dauer einer vollständigen Schwingung, mit

$$T = \frac{1}{f} \quad (2.2)$$

- **Phase** ϕ : die Anfangslage der Schwingung im Zyklus, angegeben in Radiant.

2.1.2 Abtastung und Quantisierung

Abtastung auch als Sampling bezeichnet dient dazu aufgrund dessen, dass das analoge Signale zu jedem Zeitpunkt existiert, wird zu bestimmten, gleichmäßigen Zeitpunkten das Signal abgetastet, gemessen. Alles dazwischen wird ignoriert. Die Anzahl der Messungen pro Sekunde ist die Abtastfrequenz f_s gemessen in Hz.

Bei der Abtastung ist die wichtigste Regel das *Nyquist – Theorem*. Sie besagt, dass mindestens doppelt so schnell abtasten wie die höchste Frequenz im Signal geben soll, sonst kommt es zu Fehlern.

$$f_s \geq 2 \cdot f_{\max} \quad (2.3)$$

[Vgl. 6, S. 521-522].

Quantisierung ist der Schritt nach der Abtastung. Die unendlich genaue Messwerte werden auf einen diskreten Zahlenwert gerundet. Sie wird durch die Bittiefe begrenzt. Durch das Runden wird ein Quantisierungsfehler erzeugt, wobei ein leises Rauschen aus dem Differenz wahrgenommen wird. Je mehr Bits umso besser die Audioqualität [Vgl. 6, S. 520].

$$x[n] = x(n \cdot T_s), \quad T_s = \frac{1}{f_s} \quad (2.4)$$

2.1.3 Signal-Rausch-Verhältnis (SNR)

SNR beschreibt das Verhältnis der Nutzleistung eines Signals zur Leistung des überlagerten Rauschens. Je höher das SNR, desto weniger wird das Nutzsignal durch Rauschen beeinträchtigt. [Vgl. 3, S. 5-6]

Das SNR ist definiert als:

$$\text{SNR} = \frac{P_{\text{Signal}}}{P_{\text{Rauschen}}} \quad (2.5)$$

Da Leistungsverhältnisse sehr große oder sehr kleine Zahlen annehmen können, wird das SNR in der Praxis logarithmisch in Dezibel (dB) angegeben:

$$\text{SNR}_{\text{dB}} = 10 \cdot \log_{10} \left(\frac{P_{\text{Signal}}}{P_{\text{Rauschen}}} \right) \quad (2.6)$$

Sind statt Leistungen die Amplituden bekannt, gilt:

$$\text{SNR}_{\text{dB}} = 20 \cdot \log_{10} \left(\frac{A_{\text{Signal}}}{A_{\text{Rauschen}}} \right) \quad (2.7)$$

Typische Orientierungswerte für die Sprachverarbeitung:

- SNR < 0 dB: Rauschen lauter als Nutzsignal
- SNR ≈ 10 dB: Sprache gerade noch verständlich
- SNR ≈ 20 dB: gute Sprachqualität
- SNR > 40 dB: Studioqualität

Für die spektrale Subtraktion ist das SNR die zentrale Bewertungsgröße: Ziel des Algorithmus ist es, das SNR des verrauschten Eingangssignals $y[n] = x[n] + d[n]$ durch Unterdrückung des Rauschspektrums $D[k]$ zu maximieren, ohne dabei das Nutzsignal $x[n]$ wesentlich zu verzerren. [3, S. 23]

Für die spektrale Subtraktion ist das Signal-Rausch-Verhältnis besonders wichtig, weil das Verfahren darauf abzielt, den Rauschanteil im gestörten Sprachsignal zu verringern und damit die Verständlichkeit der Sprache zu verbessern. Die Qualität der Methode lässt sich deshalb wesentlich daran beurteilen, wie stark sich das Verhältnis zwischen Nutzsignal und Rauschen durch die spektrale Bearbeitung verbessert.

2.1.4 Digitale Audiosignale als Sample-Folgen

Nach Abtastung und Quantisierung liegt das Audiosignal als diskrete Sample-Folge vor:

$$x[n] = x(n \cdot T_s), \quad n = 0, 1, \dots, N - 1 \quad (2.8)$$

Für die Verarbeitung wird die Folge in überlappende Frames der Länge N unterteilt um den Verlust von Signalen zu vermeiden:

$$x_\lambda[n] = x[n + \lambda \cdot H] \quad (2.9)$$

wobei λ den Frame-Index und H die Hop Size bezeichnen. Diese blockbasierte Struktur ist die direkte Grundlage der Kurzzeit-Fouriertransformation in Abschnitt 2.3.3 [Vgl. 3, S. 32-39].

Für die spektrale Subtraktion ist diese Darstellung als diskrete Sample-Folge notwendig, weil das Signal nicht kontinuierlich, sondern blockweise verarbeitet wird. Erst durch die Zerlegung in kurze Frames kann später für jeden Zeitabschnitt ein separates Kurzzeitspektrum berechnet und bearbeitet werden.

2.2 Frequenzanalyse mit DFT und FFT

Die Frequenzanalyse ist eine zentrale Grundlage der digitalen Sprachsignalverarbeitung, weil sie ein Signal nicht nur als zeitlichen Verlauf, sondern auch als Zusammensetzung einzelner Frequenzanteile beschreibt.

2.2.1 Zeitbereich und Frequenzbereich

Im Zeitbereich wird ein digitales Signal als Amplitude über der Abtastzeit beziehungsweise über der Abtastnummer dargestellt. Diese Darstellung beschreibt also, wie sich der Signalwert im Verlauf der Zeit ändert. [Vgl. 8, S. 91]

Im Frequenzbereich wird dasselbe Signal dagegen durch sein Spektrum beschrieben, also durch die Amplituden seiner enthaltenen Frequenzkomponenten. In dieser Darstellung liegt die Frequenz auf der x-Achse, in Hertz, während die y-Achse die Signalamplitude oder Spektralamplitude angibt. [Vgl. 8, S. 91]

Tan und Jiang zeigen diesen Zusammenhang bereits zu Beginn des Buches anhand einfacher Audiosignale: Ein 1000-Hz-Signal erscheint im Zeitbereich als periodische Schwingung, während im Frequenzbereich ein deutliches Maximum bei 1000 Hz sichtbar wird. Für ein Signal mit zwei Frequenzanteilen entstehen entsprechend zwei spektrale Peaks, etwa bei 1000 Hz und 3000 Hz. [Vgl. 8, S. 4]

Der Wechsel in den Frequenzbereich ist notwendig, weil viele Verarbeitungsaufgaben dort leichter interpretierbar und effizienter durchführbar sind.

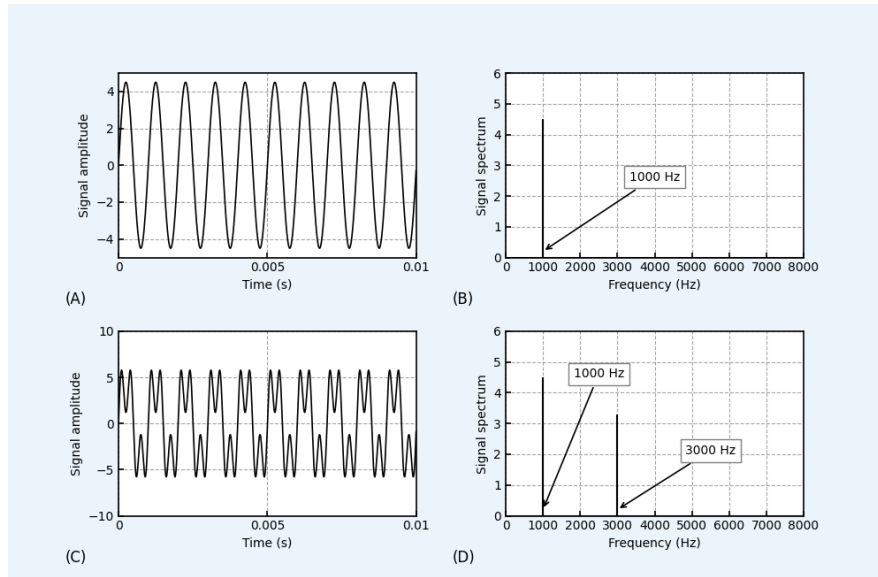


Abbildung 2.1: Zeit- und Frequenzbereich Darstellung. Visualisierung generiert durch KI (Gemini, Version 3.1)

Für die spektrale Subtraktion ist der Wechsel in den Frequenzbereich entscheidend, weil sich Sprache und Rauschen dort als spektrale Anteile beschreiben und voneinander trennen lassen. Das Verfahren arbeitet deshalb nicht direkt auf dem Zeitsignal, sondern auf den Frequenzkomponenten kurzer Sprachsegmente. [Vgl. 1, S. 113-117]

2.2.2 Diskrete Fourier-Transformation

Die mathematische Grundlage für den Übergang vom Zeitbereich in den Frequenzbereich ist die diskrete Fourier-Transformation, kurz DFT. Sie ordnet einer endlichen Folge von Abtastwerten $x[n]$ ein Spektrum $X[k]$ zu, das die Frequenzanteile des Signals beschreibt. [Vgl. 8, S. 91]

Die DFT ist definiert durch

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (2.10)$$

wobei N die Anzahl der Signalwerte, n den Zeitindex und k den Frequenzindex bezeichnet. [Vgl. 8, S. 97]

Der komplexe Exponentialterm kann auch in der Kurzschreibweise W_N^{kn} mit dem Twiddle-Faktor W_N geschrieben werden. [Vgl. 8, S. 97] Jeder DFT-Koeffizient $X[k]$ ist im Allgemeinen eine komplexe Zahl. Er beschreibt, wie stark die zum Index k gehörende Frequenzkomponente im Signal vertreten ist und welche Phasenlage sie besitzt. [Vgl. 8, S. 137] Die inverse Transformation rekonstruiert aus dem Spektrum wieder das

Zeitsignal. Tan und Jiang geben die inverse DFT in der Form

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot W_N^{-kn} \quad (2.11)$$

an und zeigen damit, dass DFT und inverse DFT ein Transformationspaar bilden. Ein praktischer Nachteil der direkten DFT ist ihr hoher Rechenaufwand. Für eine Folge der Länge N ergibt sich bei direkter Berechnung eine Komplexität von $O(N^2)$. [Vgl. 8, S. 129]

Die DFT bildet damit die mathematische Grundlage der spektralen Subtraktion. Sie liefert für jedes Frame die Frequenzbins, in denen später der geschätzte Rauschanteil vom gestörten Sprachspektrum subtrahiert werden kann.

2.2.3 Fast Fourier Transformation

Da die direkte Berechnung der diskreten Fourier-Transformation (DFT) bei großen Signallängen mit einem hohen Rechenaufwand verbunden ist, wird in der Praxis die Fast Fourier Transformation (FFT) eingesetzt. Die FFT ist ein effizienter Algorithmus zur Berechnung der DFT-Koeffizienten und reduziert die Anzahl der erforderlichen komplexen Multiplikationen erheblich [Vgl. 8, S. 127-130].

Die Effizienz der FFT beruht darauf, dass die Berechnung der DFT nicht mehr unmittelbar aus ihrer Summendefinition erfolgt, sondern in kleinere Teilprobleme zerlegt wird. Tan und Jiang behandeln hierzu insbesondere radix-2-Algorithmen in den Varianten Decimation-in-Frequency und Decimation-in-Time. Dabei wird eine N -Punkt-DFT schrittweise in kleinere DFTs zerlegt, wodurch sich die Berechnung deutlich effizienter durchführen lässt [Vgl. 8, S. 127-134].

Für eine Datenlänge von N benötigt die direkte DFT insgesamt N^2 komplexe Multiplikationen. Die FFT reduziert diesen Aufwand auf

$$\frac{N}{2} \log_2 N \quad (2.12)$$

komplexe Multiplikationen [Vgl. 8, S. 129].

Der theoretische Beschleunigungsfaktor der FFT gegenüber der direkten DFT ergibt sich damit zu

$$\frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N} \quad (2.13)$$

Für typische Signallängen zeigt sich daraus der in Tabelle 2.1 dargestellte Geschwindigkeitsvorteil.

Ein anschauliches Beispiel geben Tan und Jiang für eine Folge mit 1024 Datenpunkten. Für die direkte DFT werden dabei 1,048,576 komplexe Multiplikationen benötigt, während die FFT nur 5,120 komplexe Multiplikationen erfordert [Vgl. 8, S. 130]. Damit wird die praktische Relevanz der FFT insbesondere für große Datenmengen und Echtzeitanwendungen deutlich.

Für die praktische Implementierung der spektralen Subtraktion ist nicht nur die DFT, sondern vor allem die FFT wichtig, weil sie die spektrale Analyse und die spätere inverse Transformation rechnerisch effizient macht. Gerade für framebasierte Sprachverarbeitung und Echtzeitanwendungen ist diese Beschleunigung entscheidend.

N	DFT (N^2)	FFT ($\frac{N}{2} \log_2 N$)	Speedup
256	65,536	1,024	$64\times$
1024	1,048,576	5,120	$205\times$
4096	16,777,216	24,576	$683\times$

Tabelle 2.1: Vergleich des theoretischen Rechenaufwands von DFT und FFT

2.2.4 Amplituden- und Leistungsspektrum

Die DFT liefert für jedes Frequenzbin einen komplexen Koeffizienten $X(k)$. Da komplexe DFT-Koeffizienten nicht direkt anschaulich dargestellt werden können, werden in der Praxis daraus Betrag und Phase bestimmt. Der Betrag bildet dabei die Grundlage für das Amplitudenspektrum, während das Quadrat des Betrags zur Berechnung des Leistungsspektrums verwendet wird [Vgl. 8, S. 103].

Das Amplitudenspektrum ist definiert als

$$A_k = \frac{1}{N} |X(k)| = \frac{1}{N} \sqrt{(\operatorname{Re}\{X(k)\})^2 + (\operatorname{Im}\{X(k)\})^2}, \quad k = 0, 1, 2, \dots, N-1 \quad (2.14)$$

Das Leistungsspektrum ergibt sich entsprechend zu

$$P_k = \frac{1}{N^2} |X(k)|^2 = \frac{1}{N^2} [(\operatorname{Re}\{X(k)\})^2 + (\operatorname{Im}\{X(k)\})^2], \quad k = 0, 1, 2, \dots, N-1 \quad (2.15)$$

In vielen praktischen Anwendungen wird statt des zweiseitigen Spektrums ein einseitiges Spektrum betrachtet. Dabei bleibt der Gleichanteil bei $k = 0$ unverändert, während die übrigen Spektralanteile bis zur Faltungsfrequenz verdoppelt werden. Tan und Jiang geben für das einseitige Amplitudenspektrum die Form

$$A_k = \begin{cases} \frac{1}{N} |X(0)|, & k = 0 \\ \frac{2}{N} |X(k)|, & k = 1, \dots, \frac{N}{2} \end{cases} \quad (2.16)$$

an [Vgl. 8, S. 103].

Die graphische Darstellung von Amplituden- und Leistungsspektrum zeigt, wie sich die Energie eines Signals auf einzelne Frequenzen verteilt. Tan und Jiang veranschaulichen dies sowohl für DFT als auch für FFT anhand von Spektralplots, in denen charakteristische Peaks an den dominanten Frequenzkomponenten sichtbar werden [Vgl. 8, S. 109–110].

Für die spektrale Subtraktion ist besonders das Amplituden- beziehungsweise Magnitudenspektrum relevant, weil im klassischen Verfahren nach Boll der geschätzte Rauschbetrag direkt vom Betrag des gestörten Sprachspektrums abgezogen wird. Das Spektrum ist damit die eigentliche Arbeitsdarstellung des Verfahrens.

2.2.5 Frequenzauflösung

Ein zentraler Parameter der Spektralanalyse ist die Frequenzauflösung. Sie beschreibt den Frequenzabstand zwischen zwei benachbarten DFT-Koeffizienten und gibt damit an, wie fein das Spektrum im Frequenzbereich aufgelöst ist [Vgl. 8, S. 101, 103].

Die Frequenzauflösung ergibt sich zu

$$\Delta f = \frac{f_s}{N} \quad (2.17)$$

wobei f_s die Abtastfrequenz und N die Anzahl der ausgewerteten Datenpunkte bezeichnet [Vgl. 8, S. 103].

Aus dieser Beziehung folgt, dass eine größere Datenlänge N zu einer besseren Frequenzauflösung führt. Tan und Jiang weisen ausdrücklich darauf hin, dass mit einer längeren Datenfolge eine feinere Auflösung im Frequenzbereich erreicht werden kann [Vgl. 8, S. 103]. Eine größere Anzahl an Datenpunkten bedeutet zugleich, dass ein längerer Signalabschnitt ausgewertet werden muss. In der Spektralanalyse wird dazu eine Beobachtungsdauer von

$$T_0 = NT \quad (2.18)$$

verwendet, wobei $T = 1/f_s$ die Abtastperiode ist. Damit verbessert eine größere Fensterlänge zwar die Frequenzauflösung, erhöht aber gleichzeitig die Länge des ausgewerteten Zeitabschnitts [Vgl. 8, S. 102].

Die Frequenzauflösung bestimmt, wie fein Sprache und Rauschen im Spektrum voneinander getrennt werden können. Für die spektrale Subtraktion ist sie deshalb wichtig, weil eine zu grobe Auflösung die präzise Schätzung und Subtraktion des Rauschanteils in einzelnen Frequenzbereichen erschwert.

2.3 Kurzzeitanalyse von Sprachsignalen

Die Sprachsignalverarbeitung kann nicht allein auf einer globalen Fourier-Analyse über das gesamte Signal beruhen, da Sprachsignale zeitlich stark variieren. Für Verfahren wie die spektrale Subtraktion ist deshalb eine Kurzzeitanalyse erforderlich, bei der Sprache in kurze, überlappte Abschnitte zerlegt und für jedes Zeitfenster separat im Frequenzbereich beschrieben wird. [Vgl. 1, S. 114, 116, 117]

2.3.1 Nichtstationarität von Sprache

Sprachsignale sind im Allgemeinen nichtstationär, da sich ihre spektralen und zeitlichen Eigenschaften fortlaufend ändern. Boll weist ausdrücklich darauf hin, dass Sprache nichtstationär ist und deshalb nur eine begrenzte zeitliche Mittelung erlaubt ist. [Vgl. 1, S. 114]

Diese Eigenschaft ist für die Analyse entscheidend, weil eine zu lange Mittelung oder eine zu große Auswertzeit zu einer Verschmierung kurzer Sprachereignisse führen kann. [Vgl. 1, S. 114, 116]

Für die spektrale Subtraktion ist die Nichtstationarität von Sprache der Grund, warum nicht das Gesamtsignal auf einmal, sondern nur kurze Abschnitte analysiert werden. Innerhalb solcher kurzen Zeitfenster kann das Signal näherungsweise lokal stationär behandelt werden, was die spektrale Bearbeitung erst praktikabel macht. [Vgl. 1, S. 113, 114]

2.3.2 Frame-basierte Verarbeitung

Um Sprache kurzzeitig analysieren zu können, wird das Signal in einzelne Frames zerlegt. Boll beschreibt dieses Vorgehen konkret so, dass die Eingangsdaten in halb überlappte Datenpuffer segmentiert und anschließend mit einer Hanning-Funktion gewichtet werden. [Vgl. 1, S. 116]

Für die saubere Rekonstruktion des Signals, verweist Boll darauf, dass bei halb überlappten Puffern und Hanning-Fensterung die summierten Fenster im Optimalfall wieder das ursprüngliche Signal. [Vgl. 1, S. 116]

Zu beachten sind die Framelänge, die Schrittweite zwischen zwei Frames und die verwendete Fensterfunktion. Im Boll-Verfahren wird bei $f_s = 8\text{ kHz}$ eine Fensterlänge von 256 Punkten und eine Verschiebung um 128 Punkte verwendet, also eine Überlappung von 50%. [Vgl. 1, S. 116]

2.3.3 Short-Time Fourier Transform

Die Short-Time Fourier Transform, kurz STFT, beschreibt ein Signal nicht mehr nur global im Frequenzbereich, sondern als zeitabhängige Folge lokaler Spektren. Praktisch wird dazu auf jedes gefensterte Frame eine Fourier-Transformation angewendet, sodass für jedes Zeitfenster ein eigenes Kurzzeitspektrum entsteht. Diese Art der Analyse entspricht genau dem Vorgehen, das Boll für die spektrale Subtraktion beschreibt: Für jedes Datenfenster wird die DFT berechnet und daraus das Magnitudenspektrum bestimmt. [Vgl. 1, S. 116]

Boll verwendet die kurzzeitigen Spektren direkt zur Darstellung von Zeit-Frequenz-Verläufen. Er beschreibt isometrische Darstellungen von Zeit gegen Frequenz und Spektralmagnitude, die aus 64 überlappten Hanning-Fenstern berechnet wurden; Als Ergebnis erhält man eine zeitlich vorlaufendes Spektrum statt einer einzigen globalen Fourier-Darstellung. [Vgl. 1, S. 117]

2.3.4 Magnitude und Phase

Das Fourier-Spektrum kann in Betrag und Phase zerlegt werden. Wobei der Betrag das Amplitudenspektrum und deren Winkel das Phasenspektrum abbildet. Tan definiert das Amplitudenspektrum über den Betrag der komplexen DFT-Koeffizienten und das Phasenspektrum über deren Winkel. [Vgl. 8, S. 103] Für die Sprachverarbeitung bedeutet das, dass jeder Frame nicht nur eine Verteilung von Beträgen über der Frequenz besitzt, sondern zusätzlich Phaseninformationen enthält.

Für die spektrale Subtraktion ist insbesondere die Magnitude entscheidend, weil im klassischen Verfahren der geschätzte Rauschbetrag vom Magnitudenspektrum des gestörten Signals abgezogen wird. Die Phase des Eingangssignals bleibt dagegen meist erhalten und wird erst bei der Rekonstruktion des Zeitsignals wieder verwendet.

2.4 Theorie der Spectral Subtraction

Wie in den vorherigen Unterkapiteln schon oft angesprochen wurde ist die Spektrale Subtraktion ein Verfahren zur Rauschminderung in Sprachsignalen. Die Grundidee

besteht darin, den geschätzten Rauschanteil im Frequenzbereich vom gestörten Sprachsignal abziehen, um die Sprache klarer und verständlicher zu machen. [Vgl. 1, S.113].

Im Folgenden wird die Grundidee, das Signalmodell, die Verarbeitung im Frequenzbereich sowie die wichtigsten Voraussetzungen und Grenzen erklärt.

2.4.1 Ziel und Grundidee des Verfahrens

Ziel der Spectral Subtraction ist es, den Rauschanteil in einem Sprachsignal zu verringern und dadurch die Qualität der Sprache zu verbessern. Dazu wird zunächst das Rauschspektrum geschätzt und anschließend vom Spektrum des gestörten Signals subtrahiert. [Vgl. 1, S.113-114].



Abbildung 2.2: Blockdiagramm Spectral Subtraction, Visualisierung generiert durch KI (Gemini, Version 3.1)

2.4.2 Additives Modell von Sprache und Rauschen

Die Methode basiert auf der Annahme, dass das beobachtete Signal aus Sprache und Rauschen zusammengesetzt ist. Im Zeitbereich gilt:

$$x(k) = s(k) + n(k) \quad (2.19)$$

Variablen:

- $x(k)$: beobachtetes, verrauschtes Signal im betrachteten Fenster
- $s(k)$: sauberes Sprachsignal
- $n(k)$: Rauschsignal
- k : Abtastindex innerhalb des betrachteten Fensters

Nach der Fourier-Transformation ergibt sich:

$$X(e^{j\omega}) = S(e^{j\omega}) + N(e^{j\omega}) \quad (2.20)$$

Variablen:

- $X(e^{j\omega})$: Spektrum des verrauschten Signals
- $S(e^{j\omega})$: Spektrum der sauberen Sprache
- $N(e^{j\omega})$: Spektrum des Rauschens
- $e^{j\omega}$: komplexe Darstellung einer Frequenzkomponente
- ω : Kreisfrequenz

Im Frequenzbereich gilt dieselbe Idee, dort setzt sich das gestörte Spektrum aus Sprachspektrum und Rauschspektrum zusammen. [Vgl. 1, S.114].

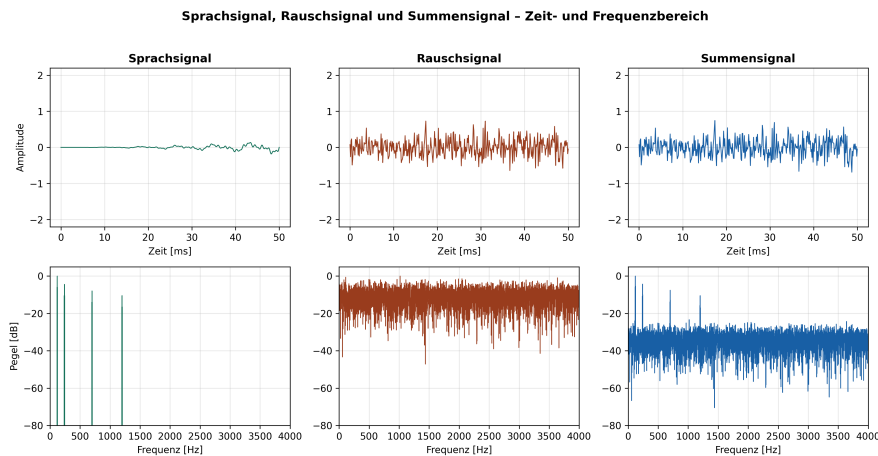


Abbildung 2.3: Sprachsignal, Rauschsignal und Summensignal im Zeit- und Frequenzbereich

2.4.3 Grundablauf der Spectral Subtraction

Der Ablauf der Spectral Subtraction lässt sich in wenige Schritte gliedern. Zuerst wird das Sprachsignal in kurze, meist überlappende Frames aufgeteilt und für jedes Frame in den Frequenzbereich transformiert.

Danach wird aus sprachfreien Abschnitten ein mittleres Rauschspektrum geschätzt. Im nächsten Schritt wird dieses geschätzte Rauschspektrum vom Magnitudenspektrum des gestörten Signals abgezogen. Wenn dabei negative Werte entstehen, werden sie auf null gesetzt. Dieser Schritt wird als Half-Wave Rectification bezeichnet. Er verhindert, dass nicht sinnvolle negative Spektralwerte weiterverarbeitet werden.

Anschließend können zusätzliche Verfahren zur Verringerung des verbleibenden Restrauschens eingesetzt werden. Zum Schluss wird aus dem bearbeiteten Spektrum wieder ein Zeitsignal rekonstruiert. Dazu wird die inverse Transformation verwendet und die einzelnen Frames werden per Overlap-Add zusammengesetzt. [Vgl. 1, S.113-118]

In vereinfachter Form besteht das Verfahren aus:

1. Segmentierung des Signals
2. Transformation in den Frequenzbereich
3. Schätzung des Rauschspektrums
4. spektrale Subtraktion
5. Begrenzung negativer Werte
6. optionale Restrauschminderung
7. Rücktransformation und Rekonstruktion

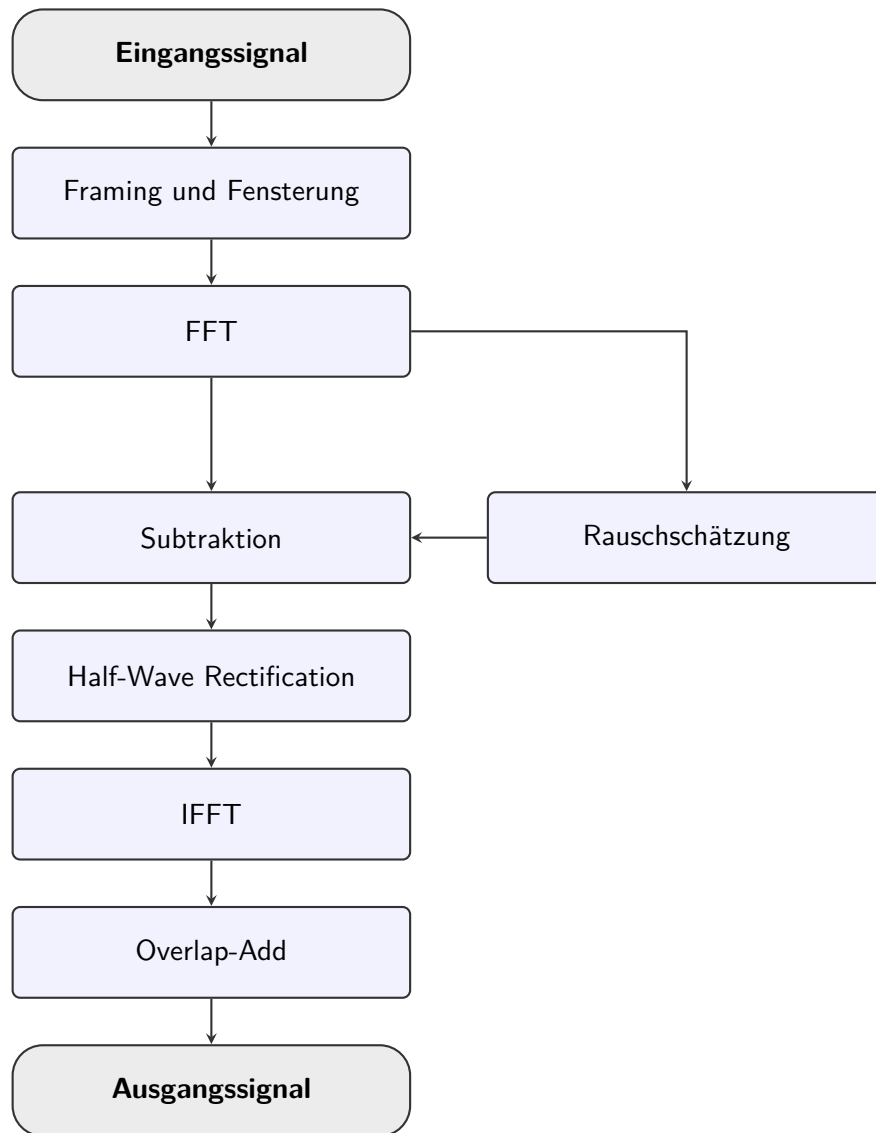


Abbildung 2.4: Ablaufdiagramm der Signalverarbeitung zur Rauschunterdrückung

2.4.4 Voraussetzungen

Die Spectral Subtraction funktioniert nicht unter allen Bedingungen gleich gut. Eine wichtige Voraussetzung ist, dass das Hintergrundrauschen wenigstens für kurze Zeit annähernd stationär bleibt. Wenn sich das Rauschen zu schnell ändert, wird die vorherige Rauschschätzung ungenau und die Subtraktion kann unwirksam oder sogar schädlich werden [Vgl. 1, S.116].

Eine weitere Voraussetzung ist, dass sprachfreie Abschnitte erkannt werden können. Nur dann kann das Rauschspektrum zuverlässig neu geschätzt werden [Vgl. 1, S.116-117].

Außerdem kann es passieren, dass bei der Subtraktion auch schwache Sprachanteile

entfernt werden. Das ist insbesondere dann problematisch, wenn an einer Frequenz die Summe aus Sprache und lokalem Rauschen kleiner ist als der geschätzte Rauschwert [Vgl. 1, S.116-117]. In diesem Fall wird nicht nur Rauschen, sondern auch nützliche Sprachinformation unterdrückt. Es besteht also immer ein Kompromiss zwischen stabiler Rauschschätzung und guter zeitlicher Auflösung.

Trotz dieser Grenzen zeigt Boll, dass das Verfahren die Sprachqualität verbessern kann und in bestimmten Anwendungen auch die Verständlichkeit erhöht [Vgl. 1, S.119]. Die Methode ist deshalb theoretisch einfach, praktisch nützlich, aber stark von den Eigenschaften des Rauschens und von einer guten Parametrierung abhängig.

2.5 Fensterung und spektrale Eigenschaften

2.5.1 Blockbildung und Randdiskontinuitäten

Für die Kurzzeitanalyse wird das Sprachsignal nicht auf einmal transformiert, sondern zunächst in einzelne, endlich lange Blöcke unterteilt. Diese Zerlegung ist erforderlich, da Verfahren wie die spektrale Subtraktion auf kurzzeitigen Spektren basieren und das Signal daher frameweise verarbeitet werden muss. Boll beschreibt genau dieses Vorgehen, indem das Sprachsignal aus halb überlappenden Eingangspuffern analysiert wird. [Vgl. 1, S. 114]

Ohne passende Gewichtung entstehen an den Blockrändern häufig Amplitudensprünge. Tan und Jiang erklären, dass diese Randdiskontinuitäten im Zeitbereich die Ursache für spektrale Verzerrungen im Frequenzbereich sind. [Vgl. 8, S. 112]

Für die spektrale Subtraktion ist die Blockbildung unvermeidbar, weil das Verfahren auf kurzzeitigen Spektren basiert. Gleichzeitig muss verhindert werden, dass künstliche Randdiskontinuitäten das Spektrum verfälschen und dadurch die spätere Rauschschätzung oder Subtraktion ungenau wird.

2.5.2 Spectral Leakage

Wie bereits erwähnt wird das Phänomen durch den Randdiskontinuitäten im Zeitbereich, als Spectral Leakage bezeichnet. Tan und Jiang zeigen, dass dabei im Spektrum zusätzliche Frequenzanteile erscheinen, die im ursprünglichen Signal nicht vorhanden sind. Die Ursache liegt in der zeitlichen Amplitudendiskontinuität des betrachteten Signalblocks. [Vgl. 8, S. 112] Sie ist für die spektrale Subtraktion problematisch, weil Energie aus einem Frequenzbereich in benachbarte Bins verschmiert werden kann. Dadurch wird die binweise Subtraktion des geschätzten Rauschanteils ungenauer und es kann leichter zu Verzerrungen der Sprache kommen. [Vgl. 1, S.113-117] Zur Verringerung dieses Effekts wird eine Fensterfunktion verwendet, deren Amplitude an beiden Enden glatt gegen Null ausläuft. Dadurch wird der Übergang zwischen den Blockrändern weicher, und die spektrale Leakage wird deutlich reduziert. [Vgl. 1, S.113-117] [Vgl. 8, S. 112]

2.5.3 Hanning- und Hamming-Fenster

Zu den gebräuchlichsten Fensterfunktionen gehören das Hanning- und das Hamming-Fenster. Tan und Jiang geben sie in der Form

$$w_{hm}(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right) \quad (2.21)$$

und

$$w_{hn}(n) = 0.5 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right) \quad (2.22)$$

an [Vgl. 8, S. 115].

Dabei gilt jeweils $0 \leq n \leq N-1$, wobei N die Fensterlänge (Anzahl der Samples) ist. Beide Fenster verringern die Amplitude an den Blockrändern und reduzieren dadurch die spektrale Leakage. [Vgl. 8, S. 112].

Für die spektrale Subtraktion ist diese Fensterwahl aus zwei Gründen zentral. Erstens verbessert sie die Qualität des geschätzten Kurzzeitspektrums vor der Subtraktion. Zweitens ermöglicht sie zusammen mit der 50%-Überlappung eine saubere Resynthese des bearbeiteten Signals. [Vgl. 1, S. 116, 117]

2.6 Verarbeitungsschritte der Spectral Subtraction

2.6.1 Rauschschätzung in sprachfreien Abschnitten

Die Spectral Subtraction benötigt zunächst eine Schätzung des Rauschens in jedem Frequenzbin. Boll definiert dafür den erwarteten Betrag des Rauschspektrums pro Frequenz als Grundlage der weiteren Verarbeitung. Diese Schätzung wird aus sprachfreien Abschnitten gewonnen, also aus Zeitbereichen, in denen nur Hintergrundrauschen und keine Sprache vorliegt [Vgl. 1, S. 116].

Boll beschreibt den mittleren Rauschbetrag als

$$P_N = E\{|N|\} \quad (2.23)$$

Wobei P_N der erwarteter oder gemittelter Betrag des Rauschspektrums ist und $E\{|N|\}$ der Erwartungswert von dem Betrag des Rauschspektrums ist.

Man betrachtet mehrere Frames ohne Sprache, berechnet für jedes Frequenzbin den Betrag des Spektrums und mittelt diese Werte über die verfügbaren sprachfreien Frames [Vgl. 1, S. 116]. In kompakter Schreibweise kann man das als praktische Näherung so beschreiben:

$$p(k) \approx \frac{1}{M} \sum_{i=0}^{M-1} |N_i(k)| \quad (2.24)$$

Diese Schreibweise legt fest, dass das Rauschspektrum durch Mittelung während sprachfreier Aktivität bestimmt wird, in eine Summenform um [Vgl. 1, S. 116]. **Variablen:**

- $p(k)$: geschätzter mittlerer Rauschbetrag im Frequenzbin k
- M : Anzahl der verwendeten sprachfreien Frames
- $N_i(k)$: Rauschspektrum des i -ten sprachfreien Frames im Frequenzbin k

- $|N_i(k)|$: Betrag dieses Spektralwerts
- k : Frequenzindex
- i : Index der sprachfreien Frames

Boll betont dabei, dass diese Schätzung nur zuverlässig ist, wenn das Rauschen lokal stationär bleibt. Ändert sich das Rauschen zu schnell, muss eine neue Schätzung durchgeführt werden, bevor erneut subtrahiert wird [Vgl. 1, S. 116].

2.6.2 Subtraktion des Rauschspektrums

Nachdem das Rauschspektrum geschätzt wurde, wird es vom gestörten Sprachspektrum abgezogen. Boll beschreibt die Methode als spektrale Schätzung, bei der die Magnitude des Rauschens durch den gemittelten Wert aus sprachfreien Abschnitten ersetzt wird [Vgl. 1, S. 114].

Für den Betrag des Ausgangsspektrums ergibt sich:

$$|\hat{S}(k)| = |X(k)| - p(k) \quad (2.25)$$

Variablen:

- $|\hat{S}(k)|$: geschätzter Betrag des bereinigten Sprachspektrums
- $|X(k)|$: Betrag des gestörten Spektrums
- $p(k)$: geschätzter Rauschbetrag im Frequenzbin k
- k : Frequenzindex

Für jedes Frequenzbin wird genau der Anteil entfernt, der vorher als mittleres Rauschen geschätzt wurde. Der Rechenschritt wird also binweise durchgeführt, nicht auf dem Gesamtsignal auf einmal [Vgl. 1, S. 116].

Boll beschreibt den Schätzer außerdem komplex, indem die Magnitude des Rauschens durch p ersetzt und seine Phase durch die Phase des gestörten Signals angenähert wird [Vgl. 1, S. 114].

$$\hat{N}(e^{j\omega}) = p(e^{j\omega})e^{j\theta_X(e^{j\omega})} \quad (2.26)$$

$$\hat{S}(e^{j\omega}) = X(e^{j\omega}) - \hat{N}(e^{j\omega}) \quad (2.27)$$

$$\hat{S}(e^{j\omega}) = X(e^{j\omega}) - p(e^{j\omega})e^{j\theta_X(e^{j\omega})} \quad (2.28)$$

Variablen:

- $\hat{N}(e^{j\omega})$: geschätztes komplexes Rauschspektrum
- $\hat{S}(e^{j\omega})$: geschätztes komplexes Sprachspektrum
- $X(e^{j\omega})$: Spektrum des gestörten Signals

- $p(e^{j\omega})$: geschätzte Rauschmagnitude als Funktion der Frequenz
- $\theta_X(e^{j\omega})$: Phase des gestörten Eingangssignals
- ω : Kreisfrequenz

2.6.3 Half-Wave Rectification

Bei der direkten Subtraktion kann es passieren, dass in einzelnen Frequenzbins negative Werte entstehen. Boll setzt diese Werte nicht weiter fort, sondern setzt sie auf null. Diesen Schritt nennt er Half-Wave Rectification [Vgl. 1, S. 116-117].

Die von Boll beschriebene Bedingung lautet inhaltlich: [Vgl. 1, S. 114-115].

- Wenn $|X(k)| \geq p(k)$ ist, bleibt nach der Subtraktion ein positiver Wert übrig.
- Wenn $|X(k)| < p(k)$ ist, wird das Ergebnis auf null gesetzt.

In heutiger kompakter Schreibweise ergibt sich damit:

$$|\hat{S}_{HR}(k)| = \max(|X(k)| - p(k), 0) \quad (2.29)$$

Diese Formel besagt Bolls Regel, dass negative Differenzen auf null gesetzt werden.

Variablen:

- $|\hat{S}_{HR}(k)|$: Ausgangsmagnitude nach Half-Wave Rectification
- $|X(k)|$: Betrag des gestörten Spektrums
- $p(k)$: geschätzter Rauschbetrag
- k : Frequenzindex
- $\max(\cdot, 0)$: Funktion, die den größeren Wert aus der Differenz und null auswählt

Boll erklärt auch die Wirkung dieses Schritts sehr konkret: Der Rauschboden wird um den geschätzten Rauschbetrag abgesenkt. Gleichzeitig besteht aber die Gefahr, dass Sprachanteile entfernt werden, wenn Sprache plus lokales Rauschen in einem Frequenzbin kleiner als der geschätzte Rauschwert sind. [Vgl. 1, S. 114-115].

2.6.4 Reduktion des Restrauschens

Auch nach der Half-Wave Rectification bleibt häufig ein Restrauschen bestehen. Boll beschreibt dieses Restrauschen als zufällig verteilte, schmale Spektralspitzen, die sich von Frame zu Frame in ihrer Amplitude ändern [Vgl. 1, S. 115]. Im Zeitbereich zeigt sich dieses Restrauschen als tonartiges, künstlich wirkendes Störsignal.

Zur Reduktion dieses Restrauschens nutzt Boll die Beobachtung, dass diese Spektralspitzen zeitlich stark schwanken. Deshalb wird in jedem Frequenzbin der aktuelle Wert durch den kleinsten Wert aus drei benachbarten Analyseframes ersetzt nur dann, wenn die aktuelle Magnitude kleiner ist als das maximale Restrauschen, das zuvor in sprachfreien Abschnitten gemessen wurde [Vgl. 1, S. 115, 117].

Formal lässt sich diese Regel wie folgt schreiben:

$$|\hat{S}_{RN,\lambda}(k)| = \begin{cases} \min \left(|\hat{S}_{HR,\lambda-1}(k)|, |\hat{S}_{HR,\lambda}(k)|, |\hat{S}_{HR,\lambda+1}(k)| \right), & \text{falls } |\hat{S}_{HR,\lambda}(k)| < R_{\max}(k) \\ |\hat{S}_{HR,\lambda}(k)|, & \text{sonst} \end{cases} \quad (2.30)$$

Variablen:

- $|\hat{S}_{RN,\lambda}(k)|$: Spektralmagnitude nach der Reduktion des Restrauschens
- $|\hat{S}_{HR,\lambda}(k)|$: Spektralmagnitude nach der Half-Wave Rectification im aktuellen Frame λ
- $|\hat{S}_{HR,\lambda-1}(k)|$: Spektralmagnitude im vorherigen Frame
- $|\hat{S}_{HR,\lambda+1}(k)|$: Spektralmagnitude im nachfolgenden Frame
- $R_{\max}(k)$: maximal gemessenes Restrauschen im Frequenzbin k während sprachfreier Aktivität
- λ : Frame-Index
- k : Frequenzindex

Die Berechnung erfolgt damit in zwei Schritten. Zuerst wird für das aktuelle Frequenzbin geprüft, ob die Magnitude kleiner als das maximal beobachtete Restrauschen ist. Nur wenn diese Bedingung erfüllt ist, wird der aktuelle Wert durch das Minimum der drei benachbarten Frames ersetzt. Andernfalls bleibt der Wert unverändert [Vgl. 1, S. 115, 117].

Der Hintergrund dieser Entscheidung ist, dass zufällige Rauschspitzen meist nicht in allen benachbarten Frames gleichzeitig groß sind. Durch die Minimumsauswahl werden solche Spitzen abgeschwächt, ohne dass hochenergetische Sprachanteile so stark geglättet werden wie bei einer einfachen Mittelung [Vgl. 1, S. 115, 117].

2.6.5 Erhalt oder Nutzung der Phaseninformation

Die Spectral Subtraction nach Boll arbeitet im Wesentlichen auf der Spektralmagnitude. Die Phase des Rauschens wird nicht separat geschätzt, sondern durch die Phase des gestörten Eingangssignals angenähert [Vgl. 1, S. 114]. Das bedeutet, dass nach der Bearbeitung der Magnitude die Phaseninformation des Eingangssignals beibehalten und für die Rekonstruktion verwendet wird.

Das komplexe Ausgangsspektrum kann entsprechend in der Form

$$\hat{S}(e^{j\omega}) = |\hat{S}(e^{j\omega})| \cdot e^{j\theta_X(e^{j\omega})} \quad (2.31)$$

geschrieben werden.

Variablen:

- $\hat{S}(e^{j\omega})$: komplexes geschätztes Ausgangsspektrum
- $|\hat{S}(e^{j\omega})|$: bearbeitete Spektralmagnitude nach Subtraktion und nachfolgenden Verarbeitungsschritten

- $\theta_X(e^{j\omega})$: Phase des gestörten Eingangssignals
- $e^{j\theta_X(e^{j\omega})}$: komplexer Phasenfaktor
- ω : Kreisfrequenz

Die Berechnung erfolgt somit so, dass zunächst nur die Magnitude verändert wird. Danach wird diese bearbeitete Magnitude wieder mit der ursprünglichen Phase des gestörten Signals kombiniert. Erst aus diesem komplexen Spektrum kann anschließend durch inverse FFT wieder ein Zeitsignal berechnet werden [Vgl. 1, S. 114, 117].

Boll beschreibt die Synthese so, dass aus der modifizierten Magnitude eine Zeitfunktion rekonstruiert wird und die einzelnen Datenfenster anschließend per Overlap-Add zum Ausgangssignal zusammengesetzt werden [Vgl. 1, S. 117]. Formal kann dieser Rekonstruktionsschritt als inverse Transformation des bearbeiteten Spektrums notiert werden:

$$\hat{s}_\lambda[n] = \text{IFFT} \left\{ \hat{S}_\lambda(k) \right\} \quad (2.32)$$

Variablen:

- $\hat{s}_\lambda[n]$: rekonstruierter Signalabschnitt im Zeitbereich für das Frame λ
- $\text{IFFT}\{\cdot\}$: inverse Fast-Fourier-Transformation
- $\hat{S}_\lambda(k)$: bearbeitetes Spektrum des Frames λ
- n : Zeitindex innerhalb des Frames
- λ : Frame-Index
- k : Frequenzindex

Die rekonstruierten Signalabschnitte werden danach überlappt addiert, sodass wieder ein zusammenhängendes Ausgangssignal entsteht [Vgl. 1, S. 117].

2.7 Rekonstruktion des bearbeiteten Signals

2.7.1 Inverse FFT

Nach der spektralen Bearbeitung muss das Signal wieder in den Zeitbereich zurückgeführt werden. Die praktische Umsetzung erfolgt über die inverse Fourier-Transformation beziehungsweise inverse FFT. Tan und Jiang nennen dafür explizit die IFFT als Standardwerkzeug zur Berechnung der inversen DFT. [Vgl. 8, S. 96-98].

Die inverse FFT ist notwendig um das Ergebnis aus dem Frequenzbereich wieder als hörbares Zeitsignal ausgeben zu können. Die spektrale Bearbeitung allein genügt nicht. Für ein verwendbares Sprachsignal muss das modifizierte Spektrum rücktransformiert werden. [Vgl. 1, S. 117]

2.7.2 Überlappende Frames

Boll analysiert Sprachdaten in halb überlappten Eingabepuffern. Konkret verwendet er bei einer Abtastrate von 8 kHz eine Fensterlänge von 256 Punkten und eine Verschiebung von 128 Punkten. Diese Überlappung ist notwendig, weil die Fensterung die Randbereiche jedes Frames abschwächt. Würde man die rekonstruierten Frames einfach ohne Überlappung hintereinander setzen, entstünden Amplitudenlücken und hörbare Verzerrungen. Durch die Überlappung tragen benachbarte Frames gemeinsam zur Rekonstruktion derselben Zeitbereiche bei. [Vgl. 1, S. 116, 117]

Für die spektrale Subtraktion ist das entscheidend, weil jeder Frame separat bearbeitet wird. Erst die überlappte Verarbeitung stellt sicher, dass aus diesen Einzelbearbeitungen wieder ein zusammenhängendes Sprachsignal entsteht. [Vgl. 1, S. 114, 117]

2.7.3 Rekonstruktion des Zeitsignals

Bei der letzten Schritt, die Resynthese, wird das Spektrum eines Frames geschätzt und vom Rauschanteil bereinigt, danach wird mit der vorhandenen Phase ein komplexes Spektrum gebildet, mit IFFT in den Zeitbereich zurücktransformiert und zuletzt per Overlap-Add mit den benachbarten Frames zusammengesetzt. [Vgl. 1, S. 114-117]

2.8 Echtzeitanforderungen

2.8.1 Latenz

Die spektrale Subtraktion arbeitet blockweise und verursacht deshalb immer eine gewisse Verzögerung. Das Sprachsignal wird zunächst in halb überlappte Datenpuffer zerlegt, im Frequenzbereich verarbeitet und anschließend wieder in ein Zeitsignal zurückgeführt. Boll verwendet dafür bei einer Abtastrate von 8 kHz eine Fensterlänge von 256 Punkten und einen Vorschub von 128 Punkten. Daran wird deutlich, dass die Latenz direkt mit der gewählten Frame-Länge und der blockweisen Verarbeitung zusammenhängt. Zusätzlich vergrößert die Mittelung mehrerer Spektren das effektive Analyseintervall. Boll nennt für drei Mittelungen einen Wert von etwa 38 ms. [Vgl. 1, S. 116]

2.8.2 Puffergröße

Die Puffergröße ist bei der spektralen Subtraktion ein wichtiger Kompromiss zwischen Frequenzauflösung, Zeitauflösung und Stabilität der Rauschschätzung. Boll erklärt, dass die Fensterlänge ungefähr doppelt so groß wie die maximal erwartete Grundperiodendauer gewählt werden sollte, um eine ausreichende spektrale Auflösung zu erreichen. In seiner Implementierung wird dafür ein 256-Punkt-Fenster verwendet. Die Fensterlänge und Mittelungsintervall müssen gegensätzliche Anforderungen erfüllen. Für eine kleine Varianz der Rauschschätzung wäre eine stärkere Mittelung günstig, für eine gute Zeitauflösung dagegen ein möglichst kurzes Analyseintervall. [Vgl. 1, S. 116]

2.8.3 Rechenzeit pro Frame

Damit ein System in Echtzeit funktioniert, muss die Berechnung eines Datenblocks (Frames) abgeschlossen sein, bevor der nächste Block zur Verarbeitung bereitsteht. Die spektrale Subtraktion ist dabei so ressourcenschonend wie eine schnelle Faltung, was sie für die effiziente digitale Signalverarbeitung besonders geeignet macht. Dieser geringe Rechenaufwand stellt sicher, dass keine Verzögerungen entstehen, die den kontinuierlichen Datenstrom unterbrechen würden. [Vgl. 1, S. 113]

2.9 Zusammenfassung

Dieses Kapitel hat die signaltheoretischen Grundlagen der spektralen Subtraktion aufgebaut. Ausgehend von den Eigenschaften digitaler Audiosignale, der Abtastung, Quantisierung und dem Signal-Rausch-Verhältnis wurden zunächst die zentralen Werkzeuge der Frequenzanalyse mit DFT und FFT sowie Amplituden-, Leistungs- und Frequenzauflösung eingeführt.

Darauf aufbauend wurde gezeigt, warum Sprachsignale aufgrund ihrer Nichtstationarität nur kurzzeitig analysiert werden können und weshalb die STFT mit framebasierter Verarbeitung, Fensterung sowie der Trennung von Magnitude und Phase die geeignete Grundlage für die spektrale Subtraktion bildet.

Abschließend wurde deutlich, dass die praktische Umsetzung zusätzlich geeignete Fensterfunktionen, Overlap-Add zur Rekonstruktion und eine echtzeitfähige Parametrierung von Frame-Länge, Überlappung und Rechenaufwand erfordert. Damit sind die wesentlichen theoretischen und technischen Voraussetzungen geschaffen, um die spektrale Subtraktion im weiteren Verlauf fundiert zu beschreiben und zu implementieren.

Kapitel 3

Faust: Functional Audio Stream

3.1 Einführung

FAUST ist eine Programmiersprache für Audio-Signalverarbeitung. Sie wurde entwickelt, um eine Alternative zu C/C++ zu bieten, mit der sich effizient Signalverarbeitungsbibliotheken, Audio-Plug-ins und eigenständige Anwendungen erstellen lassen. [Vgl. 5, S. 1]

Anders als visuelle Sprachen wie Max oder Pure Data ist FAUST eine kompilierte Sprache. Der FAUST-Compiler übersetzt FAUST-Programme direkt in C++-Code. Dieser Code ist lauffähig, selbstständig und benötigt keine externe Laufzeitumgebung. Er arbeitet deterministisch und mit konstantem Speicherverbrauch. [Vgl. 5, S. 2]

Die Besonderheit von FAUST liegt in der formalen Semantik: Jedes Programm beschreibt eine mathematische Funktion, die Eingangssignale in Ausgangssignale transformiert. Der Compiler übersetzt nicht den Programmtext, sondern die dahinterstehende Funktion. Dadurch können zwei syntaktisch unterschiedliche, aber semantisch äquivalente Programme denselben Code erzeugen. [Vgl. 5, S. 2]

3.2 Programmmodell

FAUST kombiniert funktionale Programmierung mit Blockdiagrammen. Digitale Signale sind diskrete Funktionen der Zeit, Signalprozessoren sind Funktionen auf diesen Signalen, und Kompositionsoperatoren setzen mehrere Signalprozessoren zu größeren Blockdiagrammen zusammen. Ein FAUST-Programm beschreibt keinen Klang, sondern einen Signalprozessor, also eine Maschine mit Eingängen und Ausgängen. Das Schlüsselwort `process` definiert diesen Signalprozessor, vergleichbar mit `main` in C. [Vgl. 5, S. 3]

```
1  process = 0;           // erzeugt Stille
2  process = _;           // kopiert Eingang auf Ausgang
3  process = +;           // addiert zwei Kanäle (Stereo zu Mono)
```

FAUST-Primitive funktionieren wie C-Funktionen, aber auf Signalen: `sin` wendet die Sinus-Funktion sampleweise an, `@` ist ein Delay-Operator mit $Y(t) = X(t - D(t))$. [Vgl. 5, S. 4]

3.3 Syntax: Blockdiagramm-Komposition

FAUST stellt fünf Kompositionsoperatoren bereit, die als Verallgemeinerung der Funktionskomposition $\circ$ zu verstehen sind: [Vgl. 5, S. 4]

Tabelle 3.1: Kompositionsoperatoren der Faust Block-Diagramm-Algebra

Operator	Bezeichnung	Beschreibung
$f \sim g$	Rekursive Komposition	Feedback-Schleife mit 1-Sample-Delay
f, g	Parallele Komposition	Zwei Signalprozessoren nebeneinander
$f : g$	Sequenzielle Komposition	Ausgang von f mit Eingang von g verbinden
$f < : g$	Split	Ein Ausgang auf mehrere Eingänge verteilen
$f > : g$	Merge	Mehrere Ausgänge auf einen Eingang zusammenführen

Beispiel für sequenzielle Komposition: `process = + : abs;` verbindet die Ausgabe einer Addition mit einem Absolutwert. [Vgl. 5, S. 4]

Beispiel für rekursive Komposition: `process = + _;` erzeugt einen Integrator mit $Y(t) = X(t) + Y(t - 1)$. [Vgl. 5, S. 4]

Ein vollständigeres Beispiel ist ein Rauschgenerator: [Vgl. 5, S. 5]

```
1  random = +(12345) ~ *(1103515245);
2  noise = random / 2147483647.0;
3  process = noise * vslider("noise[style:knob]", 0, 0, 100, 0.1) / 100;
```

[Vgl. 5, S. 5]

Die erste Zeile nutzt eine rekursive Komposition mit einem 1-Sample-Delay, um eine Pseudo-Zufallszahl gemäß der Formel $R(t) = 12345 + 1103515245 \cdot R(t - 1)$ zu berechnen. Die weiteren Zeilen skalieren diesen Wert in den Bereich $[-1, 1]$ und fügen einen interaktiven Lautstärkeregler hinzu, um das resultierende Signal zu steuern.

3.4 Semantik

FAUST basiert auf einer denotationale Semantik, sie beschreibt die Bedeutung eines Programms, indem sie es direkt als mathematisches Objekt definiert, in diesem Fall als eine Funktion f , die Eingangssignale in Ausgangssignale umwandelt. Anstatt den Ablauf der Berechnung Schritt für Schritt zu beschreiben, wird das Programm als ein fertiger Signalprozessor betrachtet, der eine mathematische Abbildung zwischen Signalfolgen darstellt. Dies ermöglicht es, komplexe Systeme durch die Kombination solcher Funktionen mathematisch präzise zu beschreiben.

3.5 Compiler-Architektur

Der FAUST-Compiler arbeitet in mehreren Schritten, bevor C++-Code erzeugt wird. [Vgl. 5, S. 10-12]

1. **Parsen.** Alle Quelldateien werden eingelesen und in eine Liste von Definitionen umgewandelt. Redefinitionen sind nicht erlaubt.

2. **Blockdiagramm-Auswertung.** Algorithmische Definitionen werden in flache, primitive Form gebracht (Normalform).
3. **Semantik entdecken.** Symbolische Signale werden durch das Blockdiagramm propagiert, um die mathematische Gleichung jedes Ausgangssignals zu erhalten. Verschiedene Blockdiagramme, die dasselbe berechnen, ergeben die gleichen Gleichungen.

Zwei äquivalente Programme:

```
1  process = /(2) : @ (10);
2  process = *(2) : @ (7) : /(4) : @ (3);
3
```

Beide ergeben $Y(t) = 0.5 \cdot X(t-10)$. Der Compiler normalisiert solche Gleichungen, um Delay-Lines zu teilen und Operationen zu vereinfachen.

4. **Typzuweisung** Jedes Signal bekommt einen Typ mit fünf Eigenschaften:
 - **Datentyp** Integer oder Float
 - **Wertebereich** Minimale und maximale Werte eines Signals
 - **Berechnungszeitpunkt** Zur Kompilier-, Initialisierungs- oder Laufzeit
 - **Geschwindigkeit** Konstant, langsam (pro Block) oder schnell (pro Sample)
 - **Parallelisierbarkeit** Ja bei unabhängigen Signalen, aber nicht bei rekursiven Abhängigkeiten

Aus Benutzersicht sind alle Signale volle Audiosignale; die Geschwindigkeitsunterscheidung dient nur der internen Optimierung.

5. **Vorkommenanalyse** Teilausdrücke werden auf Wiederverwendbarkeit geprüft. Häufig verwendete Ausdrücke werden in Cache-Variablen ausgelagert.

3.6 Codegenerierung

FAUST bietet drei Codegenerierungsstrategien, die aufeinander aufbauen: [Vgl. 5, S. 12-20]

3.6.1 Skalarer Modus

Ein einzelner Loop berechnet sampleweise die Ausgänge. Für jede Addition $E_1 + E_2$ wird der Code von E_1 und E_2 generiert und mit einem $+$ verbunden. Wird das Ergebnis mehrfach verwendet, wird eine Zwischenspeicher-Variable angelegt.

```
1  // Stereo zu Mono: kein Caching nötig
2  process = +;
3  $\rightarrow$ output0[i] = input0[i] + input1[i];
4
5  // Split: Ergebnis wird zwischengespeichert
6  process = + <: _,_;
7  $\rightarrow$ float fTemp0 = input0[i] + input1[i];
8  output0[i] = fTemp0;
9  output1[i] = fTemp0;
```

[Vgl. 5, S. 13]

3.6.2 Vektor-Modus

Moderne C++-Compiler nutzen SIMD-Instruktionen für Vektoroperationen (theoretischer Speedup: 4x). Damit diese Vektorisierung funktioniert, darf der Code nicht zu komplex verschachtelt sein.

Der Vektor-Modus teilt den einen großen Loop in mehrere kleine, einfache Loops auf, die über Vektoren kommunizieren. Komplexe oder gemeinsam benutzte Ausdrücke bekommen eigene Loops. Das Ergebnis ist ein gerichteter Graph (DAG) aus Schleifen.

3.6.3 Parallel-Modus

Der Parallel-Modus fügt OpenMP-Direktiven in den Vektor-Code ein, um unabhängige Schleifen parallel auf mehreren CPU-Kernen auszuführen. Der Algorithmus: Der topologisch sortierte Schleifengraph wird in Ebenen eingeteilt. Schleifen derselben Ebene sind unabhängig und können parallel laufen. Für jede Ebene erzeugt der Compiler eine `#pragma ompsections`-Direktive. Enthält eine Ebene nur eine Schleife, wird geprüft, ob diese parallelisierbar ist (keine rekursiven Abhängigkeiten). [Vgl. 5, S. 16-21]

3.6.4 Generierte Code

Der erzeugte C++-Code ist eine Klasse mit diesen Kernmethoden:

- `getNumInputs()` / `getNumOutputs()` - Anzahl Ein-/Ausgänge
- `init(int samplingFreq)` - Initialisierung
- `buildUserInterface(UI*)` - GUI-Beschreibung
- `compute(int count, float** input, float** output)` - Signalverarbeitungsmethode

3.7 Backends und Architekturdateien

FAUST trennt die Signalverarbeitungslogik von der Einbettung in Audio-Systeme. Dazu dienen Architekturdateien, die als Wrapper zwischen dem erzeugten C++-Code und dem Zielsystem stehen. Ein einziges FAUST-Programm kann so ohne Änderung für alle diese Formate [Vgl. 5, Tabelle 3, S. 7] kompiliert werden. Neue Architekturdateien lassen sich einfach hinzufügen. [Vgl. 5, S. 7]

3.8 Performance

Benchmarks mit drei Testprogrammen auf verschiedenen Hardware-Konfigurationen zeigen: [Vgl. 5, S. 22-28]

- **Einfache Programme** (z. B. Signal kopieren): Der skalare Modus ist dem Vektor-Modus ebenbürtig. Parallele Versionen sind langsamer, da die Speicherbandbreite bereits ausgelastet ist. [Vgl. 5, S. 22]

- **Programme mit mittlerem Parallelismus** (z. B. Reverb): ICC zeigt moderate Speedups bei Vektorisierung und Parallelisierung. [Vgl. 5, S. 22-23]
- **Programme mit hohem Parallelismus** (z. B. 32 parallele Strings): Deutliche Performance-Steigerung vom skalaren über den vektorisierten zum parallelen Modus. [Vgl. 5, S. 23]

Die Wirksamkeit hängt also stark vom jeweiligen Programm ab.

3.9 Zusammenfassung

FAUST ist eine kompilierte Sprache für Audio-Signalverarbeitung mit klarer formaler Semantik. Der Compiler erzeugt effizienten, eigenständigen C++-Code, der mit handgeschriebenem Code konkurrieren kann. Die drei Codegenerierungsstrategien (skalar, vektorisiert, parallel) und die Vielzahl von Architekturdateien machen FAUST flexibel für unterschiedliche Einsatzzwecke.

Kapitel 4

Mathematical Elements, Equations and Algorithms

Kapitel 5

Using Literature and other Resources

Kapitel 6

Printing the Manuscript

Kapitel 7

Closing Remarks

Anhang A

Technical Details

Anhang B

Supplementary Materials

List of supplementary data submitted to the degree-granting institution for archival storage (in ZIP format).

B.1 PDF Files

Pfad: /

thesis.pdf Master/Bachelor thesis (complete document)

B.2 Media Files

Pfad: /media

*.ai, *.pdf Adobe Illustrator files
*.jpg, *.png raster images
*.mp3 audio files
*.mp4 video files

Anhang C

Questionnaire

Anhang D

LaTeX Source Code

Quellenverzeichnis

Literatur

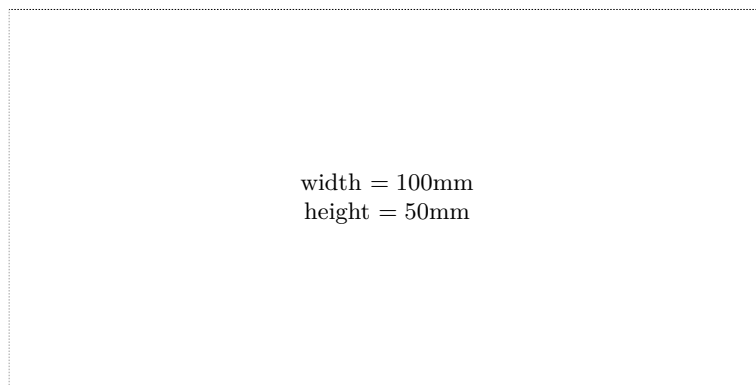
- [1] Steven Boll. „Suppression of acoustic noise in speech using spectral subtraction“. *IEEE Transactions on acoustics, speech, and signal processing* 27.2 (1979), S. 113–120 (siehe S. 2, 7, 10–22).
- [2] Stéphane Letz, Yann Orlarey und Dominique Fober. „Work Stealing Scheduler for Automatic Parallelization in Faust“. In: *Proceedings of Linux Audio Conference*. Hrsg. von LAC. Utrecht, Netherlands, 2010. URL: <https://hal.science/hal-02158924> (siehe S. 1).
- [3] Philipos C. Loizou. *Speech Enhancement: Theory and Practice*. 2. Aufl. CRC Press, 2013 (siehe S. 5, 6).
- [4] Romain Michon und Julius O. Smith. „FAUST-STK: A Set of Linear and Nonlinear Physical Models for the FAUST Programming Language“. In: *Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11)*. Paris, France, Sep. 2011 (siehe S. 2).
- [5] Yann Orlarey, Dominique Fober und Stéphane Letz. „FAUST : an Efficient Functional Approach to DSP Programming“. In: *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC*. Hrsg. von Editions DELATOUR FRANCE. 2009, S. 65–96. URL: <https://hal.science/hal-02159014> (siehe S. 1, 2, 23–27).
- [6] T. Rossing. *Springer Handbook of Acoustics*. Springer Handbook of Acoustics. Springer New York, 2007. URL: https://books.google.at/books?id=4ktVwGe_dSMC (siehe S. 4, 5).
- [7] Nicolas Scaringella u. a. „Automatic vectorization in Faust“. In: *Actes des Journées d’Informatique Musicale JIM2003*. Hrsg. von JIM. Montbeliard, France, 2003. URL: <https://hal.science/hal-02158949> (siehe S. 1).
- [8] L. Tan und J. Jiang. *Digital Signal Processing: Fundamentals and Applications*. Academic Press, 2018. URL: <https://books.google.at/books?id=MxlxDwAAQBAJ> (siehe S. 6–11, 15, 16, 20).

Online-Quellen

- [9] GRAME - Centre National de Création Musicale. *Faust Manual*. 2025. URL: <https://faustdoc.grame.fr/manual/introduction/> (besucht am 20.10.2025) (siehe S. 1).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —